

KuroAI

Generative Manga Microservices Pipeline

An enterprise-grade, AI-driven storytelling platform serving as a comprehensive MVP demonstrating full-stack engineering, microservices architecture, LLM scene planning, and Stable Diffusion panel rendering.

Author: Sachin Pandey

Platform Architect & Full-Stack Engineer

Date: May 2026

Version: 1.0.0 (MVP)

1. Project Overview & Objectives

KuroAI is a robust, highly scalable web application that transforms textual narratives into breathtaking visual manga panels using Large Language Models (LLMs) and Stable Diffusion. This repository serves as a Minimum Viable Product (MVP) showcasing production-ready software engineering principles. It was designed to demonstrate expertise across modern technology stacks, specifically aligning with full-stack enterprise development requirements.

Core Project Objectives:

- AI Storytelling:** Seamlessly map user-defined creative premises to multi-panel manga storyboard layouts.
- Visual Consistency:** Enforce strict character feature details (hair style, eyes, outfits) and artistic linework style across multiple generated frames.
- Service Decoupling:** Establish isolated API service zones, ensuring frontend presentation, backend business logic, and heavy compute AI pipelines are decoupled.
- Extensibility:** Design clean adapters for LLMs (Ollama/Mistral) and diffusion systems, facilitating future vendor swaps or model upgrades.

2. System Architecture & Tech Stack

KuroAI runs on a modern, decoupled microservices model. Instead of binding heavy machine learning calculations directly to the application layer, all generative tasks are routed through a secure python-based microservice.

Component Breakdown:

- React & TS Client:** Modern frontend designed with Vite. Implements Framer Motion and custom CSS variables for premium visual responsiveness and interactive canvas tools.
- Spring Boot Backend:** Serves as the gateway layer. Implements stateless JWT authentication, secures endpoints, and proxies generation payload calls to the python AI microservice.
- FastAPI Service:** Dedicated Python service running the AI orchestration engine. Integrates Hugging Face's Diffusers pipeline and local LLM clients.

Service Layer	Technology Stack	Key Features
Frontend Presenter	React.js, TypeScript, Vite, Framer Motion	Interactive canvas panel, glassmorphism UI
Security & Gateway Backend	Java 17, Spring Boot 3, JPA, H2	JWT filters, endpoint proxying, schema validation
AI Generation Service	Python 3.9+, FastAPI, Uvicorn	Local model load, CUDA memory optimization

3. Under the Hood: The AI Generation Pipeline

The core generation request triggers a deterministic multi-stage sequence orchestrated by Python's KuroAIOrchestrator class:

Stage 1: Storyboard Planning

The system forwards the user's core story prompt to a StoryPlannerLLM instance. This class interfaces with Ollama (running the mistral model) to parse and output a structured JSON schema. The schema defines four consecutive scenes, each containing action sequences, speaker dialogues, appropriate visual lighting moods, and camera angles.

Stage 2: Style & Composition Control

Rather than using unstructured prompts, the SceneDecomposer merges the output storyboard scenes with design specifications detailed in configs/visual_style.yaml. This system injects high-quality negative prompt anchors (distorted anatomy, bad proportions) and positive artistic anchors (high-contrast black-and-white manga, clean lineart, heavy dramatic shading).

Stage 3: Enforcing Character Consistency

To ensure that the characters remain consistent across panels, the CharacterManager assigns a deterministic integer seed generated from the character's name hash. When generating scenes containing the character (e.g., Akira the Hero), the StabilityImageAdapter forces the Stable Diffusion generator to use this fixed seed. This produces matching facial ratios, hairstyles, and outfits under different setups.

Stage 4: Rendering & Layout Synthesis

Once the Stable Diffusion pipeline generates the individual panel images, the MangaRenderer utilizes the Pillow library to dynamically overlay speech bubbles and nameplates based on the storyboard's dialogue array. The PageComposer subsequently assigns panel sizing (wide, standard, tall) for perfect canvas layouts on the web client.

4. Future Production-Level Scale Roadmap

To scale KuroAI into an enterprise SaaS hosting thousands of active users, the platform architecture is prepared to incorporate the following production features:

- **DevSecOps CI/CD Pipelines** Implement automated testing and formatting checks via GitHub Actions, combined with static analysis security testing (SAST) to secure code layers.
- **Event-Driven AI Queue (Kafka)** Handle HTTP POST requests from model run cycles. Using Kafka or RabbitMQ, jobs are queued instantly and executed asynchronously, improving throughput.
- **Distributed Caching (Redis)** Cache the popular model prompts and images to dramatically lower GPU computation expenses and page load delays.
- **Metadata Versioning Control** Implement a specialized Git-like versioning backend allowing creators to branch, edit, and rollback panel manipulations and dialogue lines securely.
- **Federated Single Sign-On (SSO)** Integrate Spring Security OAuth2 configurations to handle Google, GitHub, and Enterprise Active Directory identity integrations.

KuroAI Architectural Blueprint Summary Document

Engineered to demonstrate modern AI/ML pipelines and decoupled microservices.

Contact & Support: Sachin Pandey | KuroAI Platform Engineering